

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



BÁO CÁO THỰC HÀNH
NHÓM NLP - K35
BÀI THỰC HÀNH SỐ 1

Thành viên nhóm

GVHD: ThS. Nguyễn Mạnh Hùng

1. Nguyễn Trọng Hiếu – 25C11042 (Nhóm trưởng)
2. Trương Lê Bảo Trân – 25C11071

TP. HỒ CHÍ MINH - 06/2026

Phân công công việc

Thành viên	Nhiệm vụ	Hoàn thành
25C11042 - Nguyễn Trọng Hiếu	Mục 1: Đọc và hiển thị ảnh (open, show) + Mục 2: Scale (resize, rotate) Viết báo cáo phần Giới thiệu, Mục tiêu, Cơ sở lý thuyết của OpenCV	100%
25C11071 - Trương Lê Bảo Trân	Mục 3: Biến đổi ảnh (grayscale, brightness, contrast) Viết báo cáo phần Thực nghiệm, Kết quả, Nhận xét, Kết luận	100%

Bảng đánh giá nội dung đã hoàn thành

Nội dung		Hoàn thành
Đọc ảnh, hiển thị ảnh	Tự cài	✓
	OpenCV	✓
	So sánh	✓
Thay đổi kích thước ảnh	Tự cài	✓
	OpenCV	✓
	So sánh	✓
Xoay ảnh	Tự cài	✓
	OpenCV	✓
	So sánh	✓
Chuyển đổi màu sắc	Tự cài	✓
	OpenCV	✓
	So sánh	✓
Điều chỉnh độ sáng	Tự cài	✓
	OpenCV	✓
	So sánh	✓
Điều chỉnh độ tương phản	Tự cài	✓
	OpenCV	✓
	So sánh	✓

Mục lục

Phân công công việc.....	i
Bảng đánh giá nội dung đã hoàn thành	ii
Mục lục.....	iii
Danh mục bảng biểu	vii
Chương 1. TỔNG QUAN BÀI THỰC HÀNH	1
1.1. Mục tiêu	1
1.2. Công cụ sử dụng.....	1
1.3. Nội dung thực hiện.....	1
Chương 2. CƠ SỞ LÝ THUYẾT	2
2.1. Ảnh số.....	2
2.2. Đọc và hiển thị ảnh	2
2.3. Thay đổi kích thước ảnh (Resize)	2
2.4. Xoay ảnh (Rotate)	2
2.5. Biến đổi màu sắc	3
2.5.1. Chuyển đổi RGB sang ảnh xám (RGB → Gray).....	3
2.5.2. Chuyển đổi RGB sang HSV (RGB → HSV).....	3
2.5.3. Chuyển đổi HSV sang RGB (HSV → RGB).....	4
2.6. Điều chỉnh độ sáng.....	4
2.7. Điều chỉnh độ tương phản.....	4
2.8. Thư viện OpenCV	5
Chương 3. THIẾT KẾ VÀ CÀI ĐẶT CHƯƠNG TRÌNH	6
3.1. Môi trường thực hiện.....	6
3.2. Đọc và hiển thị ảnh	6
3.2.1. Phương pháp thủ công	6
3.2.2. Phương pháp sử dụng OpenCV	7
3.2.3. So sánh hai phương pháp	8

3.3.	Thay đổi kích thước ảnh (Resize)	8
3.3.1.	Resize ảnh bằng thuật toán Nearest Neighbor	8
3.3.2.	Phương pháp sử dụng OpenCV	9
3.3.3.	So sánh hai phương pháp	10
3.4.	Xoay ảnh (Rotate)	11
3.4.1.	Xoay ảnh bằng phương pháp Inverse Mapping	11
3.4.2.	Phương pháp sử dụng OpenCV	12
3.4.3.	So sánh hai phương pháp	13
3.5.	Biến đổi màu sắc	14
3.5.1.	Phương pháp thủ công	14
3.5.2.	Phương pháp OpenCV	16
3.5.3.	So sánh phương pháp thủ công và OpenCV	17
3.6.	Điều chỉnh độ sáng	19
3.6.1.	Phương pháp thủ công	19
3.6.2.	Phương pháp OpenCV	20
3.6.3.	So sánh hai phương pháp	21
3.7.	Điều chỉnh độ tương phản	23
3.7.1.	Phương pháp thủ công	23
3.7.2.	Phương pháp OpenCV	24
3.7.3.	So sánh hai phương pháp	25
3.8.	Nhận xét	26
TÀI LIỆU THAM KHẢO		27

Danh mục hình ảnh

Hình 1. Ảnh gốc và các kênh màu Red, Green, Blue được tách bằng phương pháp thủ công	7
Hình 2. Ảnh gốc và các kênh màu Red, Green, Blue được tách bằng OpenCV.....	8
Hình 3. Kết quả thay đổi kích thước ảnh bằng thuật toán Nearest Neighbor	9
Hình 4. Kết quả thay đổi kích thước ảnh bằng phương pháp OpenCV	10
Hình 5. So sánh kết quả Resize ảnh bằng phương pháp thủ công và OpenCV	10
Hình 6. Kết quả xoay ảnh với các góc khác nhau bằng phương pháp thủ công.....	12
Hình 7. Kết quả xoay ảnh với các góc khác nhau bằng OpenCV.....	13
Hình 8. So sánh kết quả xoay ảnh 60° giữa phương pháp Manual và OpenCV.....	13
Hình 9. So sánh kết quả và thời gian thực hiện phép xoay ảnh 60° giữa Manual và OpenCV	14
Hình 10. Kết quả chuyển đổi giữa các không gian màu RGB, Gray và HSV bằng phương pháp thủ công.....	14
Hình 11. Kết quả chuyển đổi giữa các không gian màu RGB, Gray và HSV bằng OpenCV	16
Hình 12. Kết quả so sánh thời gian thực thi và số pixel khác nhau giữa phương pháp thủ công và OpenCV.....	18
Hình 13. Ảnh so sánh kết quả chuyển đổi RGB → Gray giữa phương pháp thủ công và OpenCV	18
Hình 14. Ảnh so sánh kết quả chuyển đổi HSV → RGB giữa phương pháp thủ công và OpenCV	19
Hình 15. Kết quả điều chỉnh độ sáng ảnh với các giá trị β khác nhau bằng phương pháp thủ công.....	20
Hình 16. Kết quả điều chỉnh độ sáng ảnh với các giá trị β khác nhau bằng phương pháp OpenCV	21
Hình 17. So sánh kết quả điều chỉnh độ sáng ($\beta = +50$) giữa phương pháp thủ công và OpenCV	22
Hình 18. Kết quả so sánh kích thước và thời gian thực thi giữa phương pháp thủ công và OpenCV	22

Hình 19. Kết quả điều chỉnh độ tương phản ảnh với các hệ số α khác nhau bằng phương pháp thủ công.....	24
Hình 20. Kết quả điều chỉnh độ tương phản ảnh với các hệ số α khác nhau bằng phương pháp OpenCV	24
Hình 21. So sánh kết quả tăng độ tương phản với hệ số $\alpha = 1.5$ giữa phương pháp thủ công và OpenCV	25
Hình 22. Kết quả so sánh kích thước và thời gian thực thi giữa phương pháp thủ công và OpenCV trong điều chỉnh độ tương phản.....	25

Danh mục bảng biểu

Bảng 1. Một số hàm OpenCV sử dụng trong bài thực hành.....	5
--	---

Chương 1. TỔNG QUAN BÀI THỰC HÀNH

1.1. Mục tiêu

Bài thực hành nhằm tìm hiểu các thao tác xử lý ảnh cơ bản thông qua việc xây dựng và thử nghiệm các chức năng trên ảnh số. Các nội dung thực hiện bao gồm:

- Đọc và hiển thị ảnh.
- Thay đổi kích thước ảnh (Scale).
- Xoay ảnh (Rotate).
- Điều chỉnh màu sắc.
- Điều chỉnh độ sáng.
- Điều chỉnh độ tương phản.

Ngoài việc tự cài đặt các thuật toán xử lý ảnh, bài thực hành còn sử dụng thư viện OpenCV để đối chiếu kết quả và đánh giá hiệu quả của các phương pháp.

1.2. Công cụ sử dụng

Các công cụ và thư viện được sử dụng trong bài thực hành gồm: Python 3.x, Jupyter Notebook, NumPy, Pillow (PIL), OpenCV (cv2), Matplotlib

1.3. Nội dung thực hiện

Bài thực hành được chia thành ba nhóm chức năng chính:

Đọc và hiển thị ảnh: Thực hiện việc mở tập tin ảnh, đọc dữ liệu ảnh và hiển thị kết quả trên màn hình.

Thay đổi kích thước và xoay ảnh: Thực hiện các phép biến đổi hình học trên ảnh như phóng to, thu nhỏ và xoay ảnh theo các góc xác định.

Biến đổi ảnh: Thực hiện các phép biến đổi trên giá trị điểm ảnh nhằm thay đổi màu sắc, độ sáng và độ tương phản của ảnh.

Các kết quả thu được sẽ được so sánh với các hàm tương ứng của thư viện OpenCV để đánh giá tính chính xác và hiệu quả của chương trình.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Ảnh số

Ảnh số là ảnh được biểu diễn dưới dạng ma trận các điểm ảnh (pixel). Mỗi pixel lưu trữ thông tin về cường độ sáng hoặc màu sắc tại một vị trí trên ảnh.

Đối với ảnh màu RGB, mỗi pixel được biểu diễn bởi ba thành phần:

- R (Red): màu đỏ
- G (Green): màu xanh lá
- B (Blue): màu xanh dương

Giá trị của mỗi kênh màu thường nằm trong khoảng từ 0 đến 255. Ví dụ, một ảnh có kích thước 512×512 sẽ được lưu dưới dạng ma trận gồm 512 hàng và 512 cột pixel.

2.2. Đọc và hiển thị ảnh

Đọc ảnh là quá trình chuyển dữ liệu từ tập tin ảnh thành ma trận pixel trong bộ nhớ. Sau khi đọc, các giá trị pixel có thể được truy xuất và xử lý bằng các thuật toán. Việc hiển thị ảnh giúp quan sát kết quả trước và sau khi áp dụng các phép biến đổi.

2.3. Thay đổi kích thước ảnh (Resize)

Resize là quá trình thay đổi chiều rộng và chiều cao của ảnh.

Mục đích của phép Resize:

- Thu nhỏ ảnh để giảm dung lượng lưu trữ.
- Phóng to ảnh để thuận tiện cho việc quan sát.
- Chuẩn hóa kích thước dữ liệu đầu vào.

Trong bài thực hành, việc Resize được thực hiện bằng cách xác định vị trí tương ứng của pixel trên ảnh gốc và gán giá trị cho pixel trên ảnh mới.

2.4. Xoay ảnh (Rotate)

Xoay ảnh là phép biến đổi hình học làm thay đổi vị trí các pixel theo một góc xác định. Tọa độ mới của điểm ảnh được tính theo công thức:

$$x' = x \cos \theta - y \sin \theta$$

và

$$y' = x \sin \theta + y \cos \theta$$

Trong đó:

- (x,y) là tọa độ ban đầu.
- (x',y') là tọa độ sau khi xoay.
- θ là góc xoay.

Phép xoay được sử dụng để thay đổi hướng của ảnh hoặc chuẩn hóa dữ liệu hình ảnh.

2.5. Biến đổi màu sắc

Biến đổi màu sắc là quá trình chuyển đổi ảnh từ không gian màu này sang không gian màu khác nhằm phục vụ cho việc hiển thị hoặc xử lý ảnh.

Trong bài thực hành, các phép biến đổi màu sắc được thực hiện gồm chuyển đổi giữa các không gian màu RGB, Grayscale và HSV.

2.5.1. Chuyển đổi RGB sang ảnh xám (RGB $\rightarrow$ Gray)

Ảnh xám chỉ lưu thông tin về cường độ sáng của mỗi pixel thay vì ba thành phần màu RGB. Giá trị mức xám được tính theo công thức:

$$Gray = 0.299R + 0.587G + 0.114B$$

Trong đó:

- R : giá trị kênh đỏ.
- G : giá trị kênh xanh lá.
- B : giá trị kênh xanh dương.

Việc chuyển đổi sang ảnh xám giúp giảm lượng dữ liệu cần xử lý và thường được sử dụng trong các bài toán thị giác máy tính.

2.5.2. Chuyển đổi RGB sang HSV (RGB $\rightarrow$ HSV)

HSV là không gian màu gồm ba thành phần:

- **H (Hue):** sắc độ màu.
- **S (Saturation):** độ bão hòa màu.
- **V (Value):** độ sáng.

Khác với RGB, không gian màu HSV tách riêng thông tin màu sắc và độ sáng, giúp việc xử lý màu trở nên thuận tiện hơn.

Trong xử lý ảnh, HSV thường được sử dụng để:

- Nhận diện màu sắc.
- Phân đoạn ảnh theo màu.
- Theo dõi đối tượng.

Ví dụ:

Màu	Hue
Đỏ	0°
Xanh lá	120°
Xanh dương	240°

2.5.3. Chuyển đổi HSV sang RGB (HSV → RGB)

Sau khi xử lý trong không gian màu HSV, ảnh thường được chuyển ngược về RGB để hiển thị trên màn hình.

Quá trình chuyển đổi HSV sang RGB sử dụng các công thức nội suy dựa trên giá trị H, S và V để tính lại ba thành phần R, G và B.

Phép chuyển đổi này giúp bảo toàn thông tin màu sắc đồng thời cho phép hiển thị ảnh trên các thiết bị sử dụng chuẩn RGB.

2.6. Điều chỉnh độ sáng

Độ sáng của ảnh được thay đổi bằng cách cộng hoặc trừ một giá trị vào tất cả các pixel. Công thức:

$$I' = I + \beta$$

Trong đó:

- I : giá trị pixel ban đầu.
- I' : giá trị pixel sau khi điều chỉnh.
- β : giá trị tăng hoặc giảm độ sáng.

Ý nghĩa:

- Nếu $\beta > 0 \rightarrow$ ảnh sáng hơn.
- Nếu $\beta < 0 \rightarrow$ ảnh tối hơn.

2.7. Điều chỉnh độ tương phản

Độ tương phản thể hiện mức độ khác biệt giữa các vùng sáng và tối trên ảnh. Công thức:

$$I' = \alpha I$$

Trong đó:

- I : giá trị pixel ban đầu.
- I' : giá trị pixel sau khi biến đổi.
- α : hệ số điều chỉnh độ tương phản.

Ý nghĩa:

- $\alpha > 1$: tăng độ tương phản.
- $0 < \alpha < 1$: giảm độ tương phản.
- $\alpha = 1$: ảnh không thay đổi.

2.8. Thư viện OpenCV

OpenCV (Open Source Computer Vision Library) là thư viện mã nguồn mở chuyên dùng cho xử lý ảnh và thị giác máy tính. Thư viện được phát triển nhằm hỗ trợ các tác vụ như đọc ảnh, biến đổi ảnh, nhận dạng đối tượng và xử lý video với hiệu năng cao.

OpenCV cung cấp nhiều hàm được tối ưu sẵn, giúp giảm thời gian cài đặt thuật toán và tăng tốc độ xử lý so với việc xây dựng thủ công từ đầu. Nhờ đó, OpenCV được sử dụng rộng rãi trong nghiên cứu và phát triển các ứng dụng xử lý ảnh.

Trong bài thực hành này, OpenCV được sử dụng để thực hiện các chức năng xử lý ảnh cơ bản và đối chiếu với kết quả của phương pháp tự cài đặt.

Một số hàm OpenCV được sử dụng trong bài thực hành được trình bày trong Bảng 1.

Bảng 1. Một số hàm OpenCV sử dụng trong bài thực hành

Hàm	Chức năng
cv2.imread()	Đọc ảnh từ tập tin
cv2.resize()	Thay đổi kích thước ảnh
cv2.rotate()	Xoay ảnh
cv2.cvtColor()	Chuyển đổi không gian màu
cv2.convertScaleAbs()	Điều chỉnh độ sáng và độ tương phản

Việc sử dụng OpenCV giúp đơn giản hóa quá trình xử lý ảnh, đồng thời tạo cơ sở để so sánh hiệu quả giữa phương pháp tự cài đặt và phương pháp sử dụng thư viện có sẵn.

Chương 3. THIẾT KẾ VÀ CÀI ĐẶT CHƯƠNG TRÌNH

3.1. Môi trường thực hiện

Chương trình được xây dựng bằng ngôn ngữ Python và thực hiện trên môi trường Jupyter Notebook. Các thư viện sử dụng gồm:

- NumPy: xử lý dữ liệu ma trận ảnh.
- Pillow (PIL): hỗ trợ đọc và thao tác với ảnh.
- OpenCV (cv2): cung cấp các hàm xử lý ảnh.
- Matplotlib: hiển thị ảnh và kết quả thực nghiệm.

3.2. Đọc và hiển thị ảnh

3.2.1. Phương pháp thủ công

Ảnh được đọc từ tập tin bằng OpenCV với hàm `cv2.imread()` và chuyển đổi từ không gian màu BGR sang RGB để hiển thị đúng màu sắc. Sau đó, chương trình thao tác trực tiếp trên mảng pixel bằng NumPy để lấy thông tin ảnh, truy xuất giá trị pixel và tách các kênh màu.

Để quan sát từng thành phần màu của ảnh, ba kênh Red (R), Green (G) và Blue (B) được tách riêng bằng cách truy cập trực tiếp vào mảng ảnh. Trong NumPy, ảnh RGB được lưu dưới dạng mảng 3 chiều (height, width, channels), do đó `img_np[:, :, 0]`, `img_np[:, :, 1]` và `img_np[:, :, 2]` lần lượt tương ứng với các kênh màu Red, Green và Blue. Các kênh này sau đó được sử dụng để tạo và hiển thị ảnh riêng cho từng màu.



Hình 1. Ảnh gốc và các kênh màu Red, Green, Blue được tách bằng phương pháp thủ công

3.2.2. Phương pháp sử dụng OpenCV

Ảnh được đọc bằng hàm ***cv2.imread()*** và lấy các thông tin cơ bản như kích thước, kiểu dữ liệu và giá trị pixel. Việc tách các kênh màu được thực hiện bằng hàm ***cv2.split()***, giúp tách ảnh thành ba kênh Blue (B), Green (G) và Red (R).

Sau đó, các ảnh riêng cho từng kênh màu được tạo bằng hàm ***cv2.merge()***. Trước khi hiển thị, ảnh được chuyển từ định dạng BGR sang RGB bằng hàm ***cv2.cvtColor()***. Vì OpenCV sử dụng định dạng màu BGR, trong khi Matplotlib sử dụng RGB. Kết quả thu được gồm ảnh gốc và ba ảnh thể hiện riêng từng kênh màu Red, Green và Blue.



Hình 2. Ảnh gốc và các kênh màu Red, Green, Blue được tách bằng OpenCV

3.2.3. So sánh hai phương pháp

Hai phương pháp đều cho kết quả giống nhau, gồm ảnh gốc và các ảnh thể hiện riêng từng kênh màu Red, Green và Blue.

Ở phương pháp thủ công, việc tách kênh màu được thực hiện bằng cách truy cập trực tiếp vào mảng ảnh thông qua NumPy. Cách làm này giúp hiểu rõ hơn cách dữ liệu ảnh RGB được lưu trữ và cách tạo ảnh từ từng kênh màu.

Ở phương pháp OpenCV, các hàm `cv2.split()` và `cv2.merge()` được sử dụng để tách và ghép các kênh màu. Nhờ đó, chương trình ngắn gọn hơn và dễ thực hiện hơn so với cách thao tác trực tiếp trên mảng ảnh.

3.3. Thay đổi kích thước ảnh (Resize)

3.3.1. Resize ảnh bằng thuật toán Nearest Neighbor

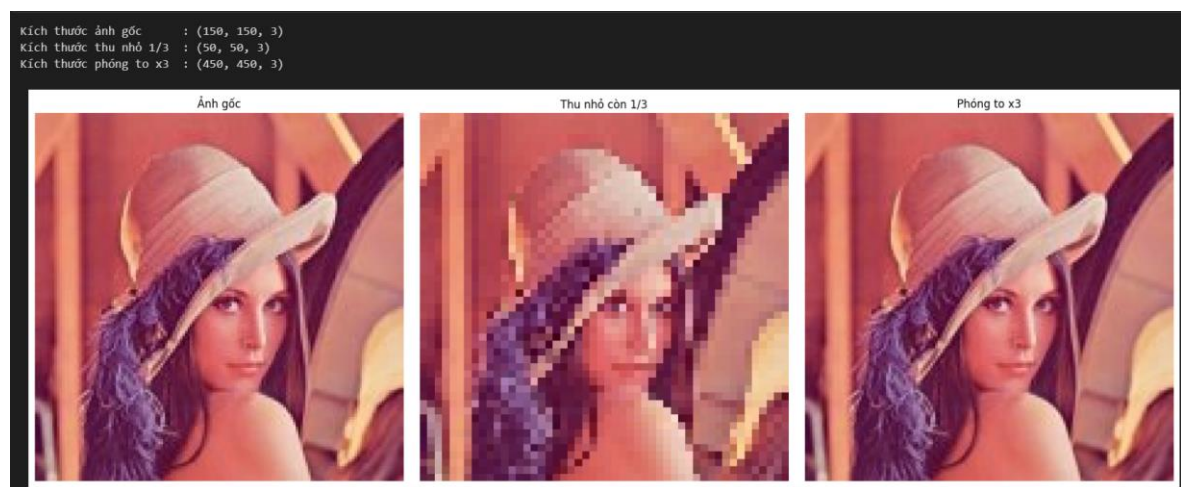
Trong phần cài đặt thủ công, việc thay đổi kích thước ảnh được thực hiện bằng thuật toán Nearest Neighbor.

Ý tưởng của thuật toán là mỗi pixel trên ảnh mới sẽ lấy giá trị từ pixel gần nhất trên ảnh gốc. Đối với mỗi vị trí (x, y) của ảnh sau khi resize, chương trình tính vị trí tương ứng (src_x, src_y) trên ảnh gốc theo công thức:

- $src_x = x / scale$
- $src_y = y / scale$

Sau đó giá trị màu tại (src_x, src_y) được sao chép sang pixel (x, y) của ảnh mới.

Thuật toán Nearest Neighbor có ưu điểm là đơn giản, dễ cài đặt và tốc độ xử lý nhanh. Tuy nhiên khi phóng to ảnh, các pixel được lặp lại nhiều lần nên ảnh thường xuất hiện hiện tượng răng cưa hoặc vỡ hình.



Hình 3. Kết quả thay đổi kích thước ảnh bằng thuật toán Nearest Neighbor

Kết quả thực nghiệm cho thấy ảnh gốc có kích thước **150 × 150 pixel**, sau khi thu nhỏ còn **50 × 50 pixel** (tỷ lệ 1/3) và sau khi phóng to đạt **450 × 450 pixel** (gấp 3 lần kích thước ban đầu).

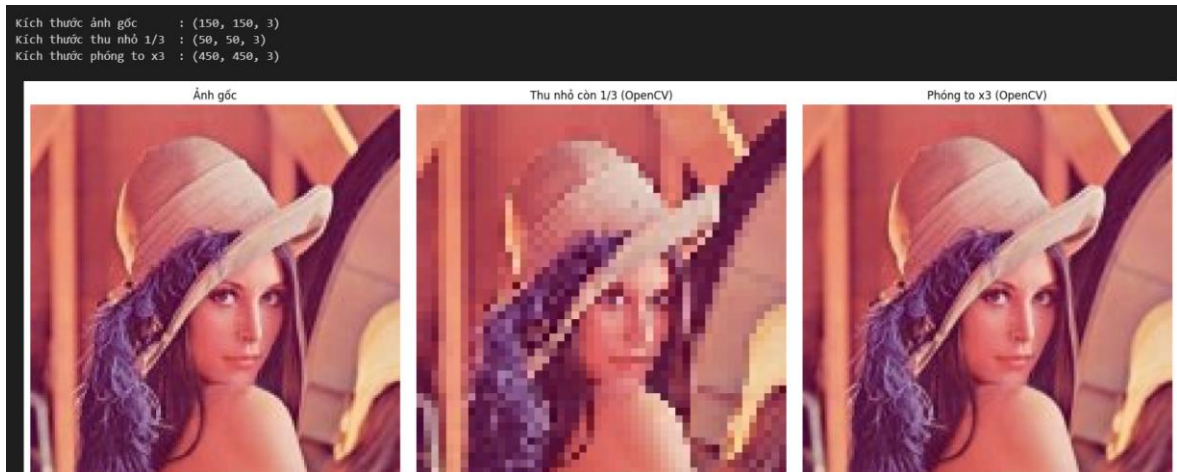
Ở ảnh **thu nhỏ 1/3**, số lượng pixel giảm đáng kể nên nhiều chi tiết của ảnh bị mất. Tuy nhiên hình dạng tổng thể của đối tượng vẫn được giữ nguyên và có thể nhận biết rõ nội dung ảnh. Do thuật toán Nearest Neighbor chỉ chọn pixel gần nhất mà không thực hiện nội suy nên quá trình thu nhỏ diễn ra nhanh nhưng chất lượng ảnh giảm.

Ở ảnh **phóng to x3**, có thể quan sát rõ hiện tượng các điểm ảnh bị phóng lớn thành các khối vuông riêng biệt. Đây là hiện tượng pixelation (răng cưa hoặc vỡ hạt) thường gặp khi sử dụng thuật toán Nearest Neighbor. Nguyên nhân là mỗi pixel của ảnh gốc được sao chép thành nhiều pixel ở ảnh mới mà không tạo ra giá trị màu trung gian giữa các điểm ảnh lân cận.

3.3.2. Phương pháp sử dụng OpenCV

Việc thay đổi kích thước ảnh được thực hiện bằng hàm **cv2.resize()** của OpenCV. Trong chương trình, ảnh được phóng to lên 3 lần và thu nhỏ còn 1/3 kích thước ban đầu thông qua các hệ số fx và fy.

Phương pháp được sử dụng là INTER_NEAREST, tương ứng với thuật toán Nearest Neighbor đã cài đặt ở phần thủ công. Với phương pháp này, mỗi pixel của ảnh mới sẽ nhận giá trị từ pixel gần nhất của ảnh gốc.



Hình 4. Kết quả thay đổi kích thước ảnh bằng phương pháp OpenCV

3.3.3. So sánh hai phương pháp

```
===== SO SÁNH KÍCH THƯỚC =====  
Manual : (450, 450, 3)  
OpenCV : (450, 450, 3)  
  
===== SO SÁNH NỘI DUNG =====  
Hai ảnh giống nhau: True  
Số phần tử khác nhau: 0  
  
===== SO SÁNH TỐC ĐỘ =====  
Manual : 0.172365 giây  
OpenCV : 0.001001 giây  
OpenCV nhanh hơn khoảng 172.17 lần
```

Hình 5. So sánh kết quả Resize ảnh bằng phương pháp thủ công và OpenCV

Kết quả thực nghiệm cho thấy cả hai phương pháp đều tạo ra ảnh đầu ra có kích thước **450 × 450 × 3**, đúng với yêu cầu phóng to ảnh lên 3 lần so với kích thước ban đầu. Điều này chứng tỏ cả hai phương pháp đều thực hiện chính xác phép biến đổi kích thước ảnh.

Kết quả kiểm tra nội dung ảnh cho thấy ảnh được tạo bởi phương pháp thủ công hoàn toàn trùng khớp với ảnh được tạo bởi hàm **cv2.resize()** của OpenCV khi cùng sử dụng thuật

toán **Nearest Neighbor**. Kết quả này xác nhận rằng thuật toán Resize được cài đặt thủ công đã hoạt động chính xác và tuân theo nguyên lý của phương pháp Nearest Neighbor.

Về tốc độ xử lý, OpenCV nhanh hơn khoảng **172.17 lần** so với phương pháp thủ công. Nguyên nhân là phương pháp Manual sử dụng các vòng lặp để duyệt từng pixel của ảnh và tự tính toán vị trí tương ứng trên ảnh gốc. Trong khi đó, OpenCV được xây dựng bằng ngôn ngữ tối ưu hiệu năng cao và sử dụng các kỹ thuật tăng tốc xử lý ảnh, giúp giảm đáng kể thời gian thực thi.

Từ kết quả trên có thể kết luận rằng phương pháp thủ công phù hợp để tìm hiểu nguyên lý hoạt động của thuật toán Resize, còn OpenCV phù hợp hơn trong các ứng dụng thực tế nhờ khả năng xử lý nhanh và hiệu quả. Mặc dù tốc độ khác nhau rất lớn, hai phương pháp vẫn cho kết quả hình ảnh hoàn toàn giống nhau khi sử dụng cùng thuật toán Nearest Neighbor.

3.4. Xoay ảnh (Rotate)

3.4.1. Xoay ảnh bằng phương pháp Inverse Mapping

Trong phần cài đặt thủ công, ảnh được xoay quanh tâm ảnh bằng phương pháp Inverse Mapping kết hợp với nội suy Nearest Neighbor. Trước tiên, chương trình xác định tâm ảnh bằng:

$$cx = (w - 1) / 2.0$$

$$cy = (h - 1) / 2.0$$

Sau đó, chương trình duyệt qua từng pixel của ảnh kết quả và tính ngược vị trí tương ứng trên ảnh gốc bằng các công thức:

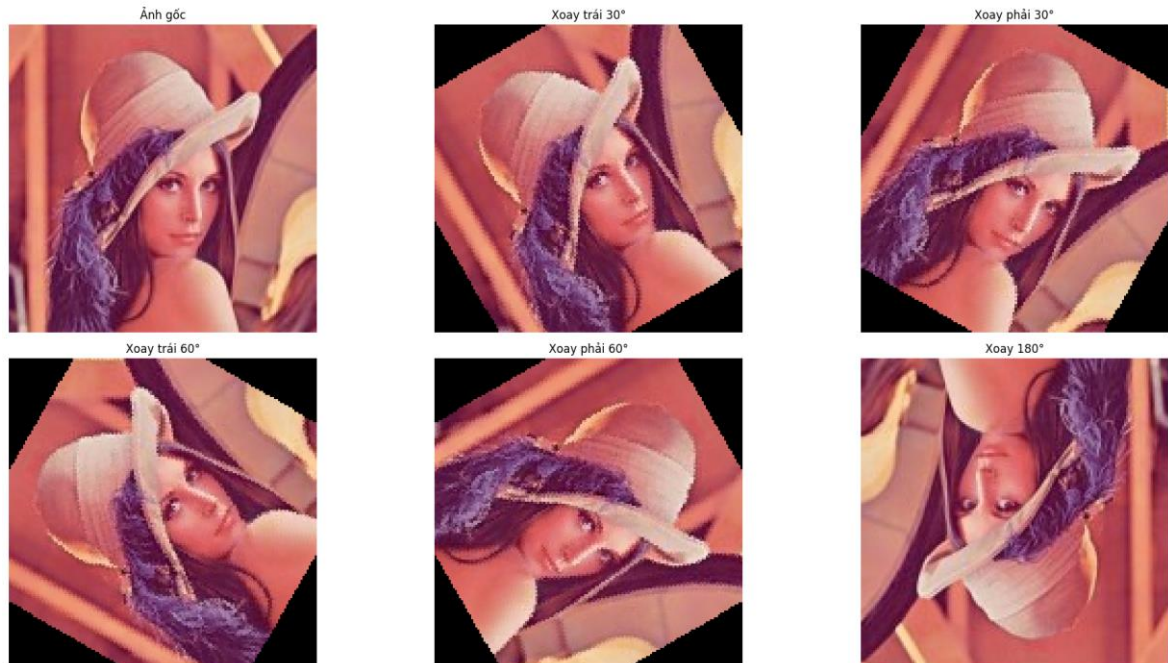
$$src_x = \cos_theta * x + \sin_theta * y + cx$$

$$src_y = -\sin_theta * x + \cos_theta * y + cy$$

Phương pháp này được gọi là Inverse Mapping vì thay vì xác định pixel trên ảnh gốc sẽ di chuyển đến đâu sau khi xoay, chương trình sẽ xét từng pixel trên ảnh kết quả và tìm vị trí cần lấy màu từ ảnh gốc. Cách tiếp cận này giúp hạn chế hiện tượng xuất hiện các vùng trống sau khi xoay ảnh.

Sau khi tìm được tọa độ trên ảnh gốc, chương trình sử dụng phép làm tròn để chọn pixel gần nhất (Nearest Neighbor), sau đó sao chép giá trị màu sang ảnh kết quả bằng câu lệnh: `result[y_new, x_new] = img[src_y, src_x]`.

Do công thức Inverse Mapping thực hiện phép quay ngược khi truy tìm vị trí trên ảnh gốc, quy ước dấu của góc xoay có thể khác với các hàm có sẵn của OpenCV. Tuy nhiên, nguyên lý xử lý của hai phương pháp là tương đương nhau và đều tạo ra ảnh được xoay quanh tâm ảnh.



Hình 6. Kết quả xoay ảnh với các góc khác nhau bằng phương pháp thủ công

3.4.2. Phương pháp sử dụng OpenCV

Trong phần này, việc xoay ảnh được thực hiện bằng các hàm có sẵn của OpenCV. Đầu tiên, chương trình xác định tâm ảnh bằng câu lệnh:

```
center = ((w - 1) / 2.0, (h - 1) / 2.0)
```

Tâm ảnh được sử dụng làm điểm xoay để ảnh quay quanh vị trí chính giữa. Tiếp theo, hàm **cv2.getRotationMatrix2D()** được sử dụng để tạo ma trận xoay:

```
M = cv2.getRotationMatrix2D(center, angle_deg, 1.0 )
```

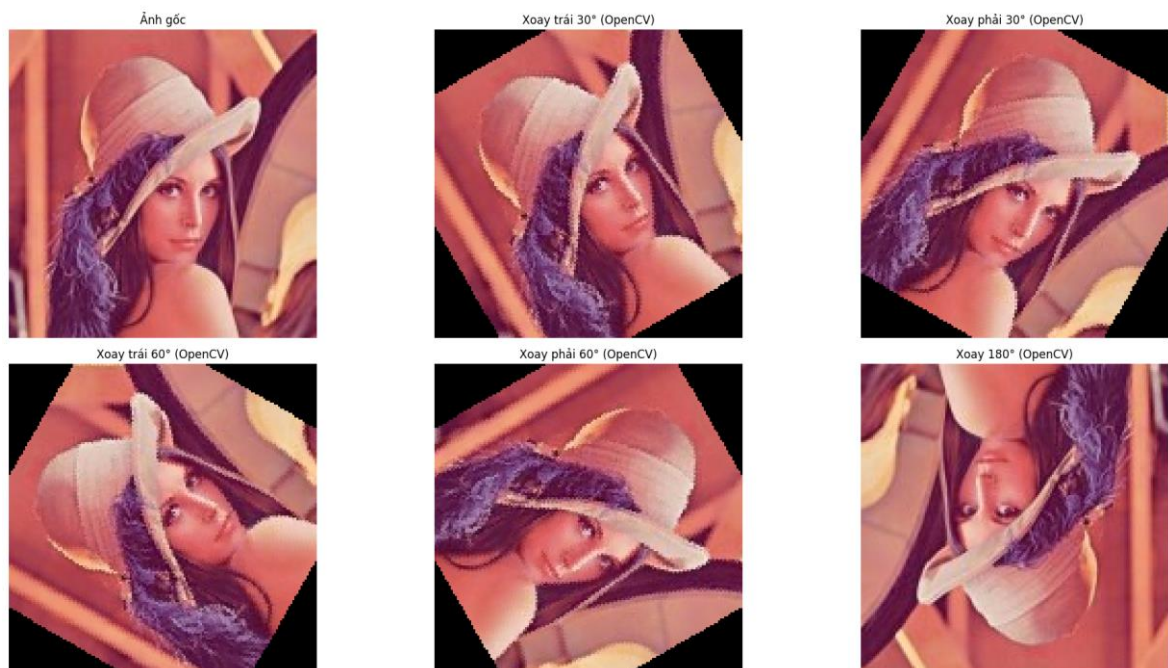
Trong đó:

- center là tâm ảnh.
- angle_deg là góc xoay.
- 1.0 là hệ số giữ nguyên kích thước ảnh.

Sau khi có ma trận xoay, chương trình thực hiện phép biến đổi bằng hàm:

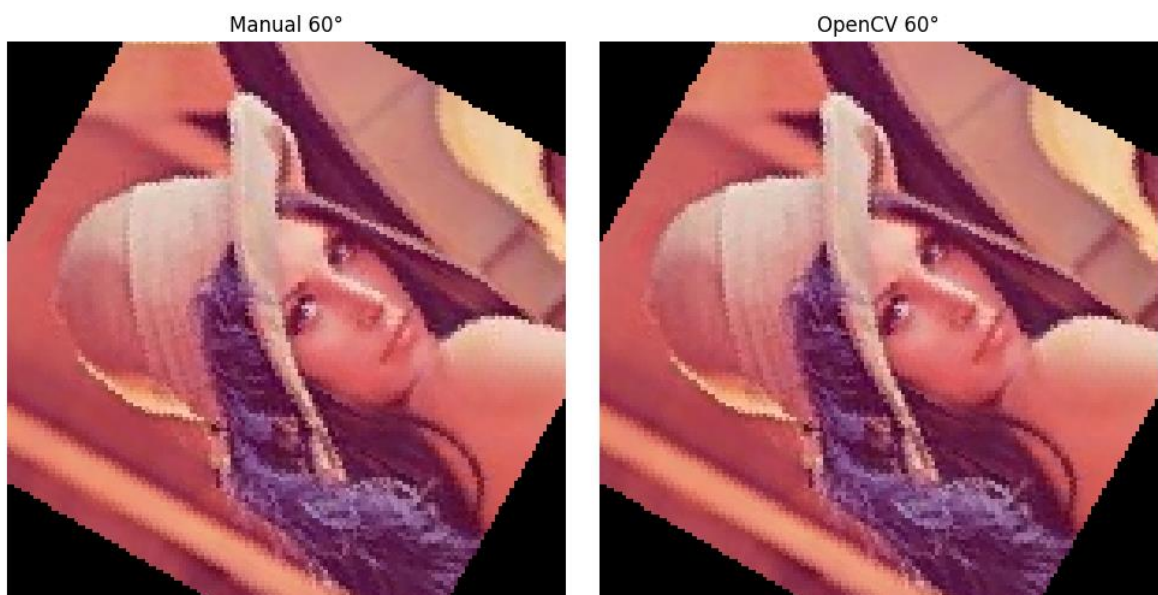
```
rotated = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_NEAREST)
```


Hàm `cv2.warpAffine()` sử dụng ma trận xoay để tính toán vị trí mới của các pixel và tạo ra ảnh sau khi xoay. Tham số `cv2.INTER_NEAREST` cho biết OpenCV sử dụng phương pháp Nearest Neighbor, tương tự như phần cài đặt thủ công.



Hình 7. Kết quả xoay ảnh với các góc khác nhau bằng OpenCV

3.4.3. So sánh hai phương pháp



Hình 8. So sánh kết quả xoay ảnh 60° giữa phương pháp Manual và OpenCV

```

===== SO SÁNH KÍCH THƯỚC =====
Manual : (150, 150, 3)
OpenCV : (150, 150, 3)

===== SO SÁNH TỐC ĐỘ =====
Manual : 0.069010 giây
OpenCV : 0.000991 giây
OpenCV nhanh hơn khoảng 69.65 lần

```

Hình 9. So sánh kết quả và thời gian thực hiện phép xoay ảnh 60° giữa Manual và OpenCV

Kết quả thực nghiệm cho thấy ảnh xoay bằng phương pháp thủ công (Manual) và OpenCV có cùng kích thước đầu ra là **(150, 150, 3)**. Điều này chứng tỏ cả hai phương pháp đều giữ nguyên kích thước khung ảnh sau khi thực hiện phép xoay.

Quan sát trực quan cho thấy ảnh thu được từ hai phương pháp gần như giống hệt nhau. Cả hai đều xoay ảnh 60° quanh tâm ảnh, giữ đúng hướng xoay và xuất hiện các vùng màu đen ở bốn góc do một phần nội dung ảnh sau khi xoay nằm ngoài phạm vi khung hình ban đầu.

Về tốc độ xử lý, có thể thấy OpenCV nhanh hơn khoảng **69.65 lần** so với phương pháp cài đặt thủ công. Nguyên nhân là do phương pháp Manual sử dụng hai vòng lặp để duyệt từng pixel của ảnh và tự tính toán tọa độ nguồn theo công thức Inverse Mapping. Trong khi đó, OpenCV sử dụng các hàm được tối ưu hóa ở mức thư viện, tận dụng các kỹ thuật xử lý hiệu năng cao nên thời gian thực thi ngắn hơn đáng kể.

Kết quả này cho thấy phương pháp thủ công phù hợp cho mục đích học tập và tìm hiểu nguyên lý hoạt động của phép xoay ảnh, còn OpenCV phù hợp cho các ứng dụng thực tế nhờ khả năng xử lý nhanh và thuận tiện hơn.

3.5. Biến đổi màu sắc

3.5.1. Phương pháp thủ công



Hình 10. Kết quả chuyển đổi giữa các không gian màu RGB, Gray và HSV bằng phương pháp thủ công

RGB → Gray

Trong ảnh RGB, mỗi pixel được biểu diễn bởi ba thành phần màu Red (R), Green (G) và Blue (B). Để chuyển ảnh màu thành ảnh xám, ba thành phần này được kết hợp thành một giá trị duy nhất thể hiện mức độ sáng của pixel.

Chương trình tách ba kênh màu bằng các câu lệnh:

```
R = img[:, :, 0]
```

```
G = img[:, :, 1]
```

```
B = img[:, :, 2]
```

Sau đó mức xám được tính theo công thức (như đã định nghĩa ở chương 2):

$$\text{gray} = (0.299 * R + 0.587 * G + 0.114 * B)$$

Các hệ số này không được chọn ngẫu nhiên mà dựa trên khả năng cảm nhận màu sắc của mắt người. Mắt người nhạy với màu xanh lá hơn màu đỏ và ít nhạy nhất với màu xanh dương, vì vậy kênh Green có hệ số lớn nhất. Kết quả cuối cùng được chuyển về kiểu dữ liệu *uint8* để lưu dưới dạng ảnh xám.

RGB → HSV

Không gian màu HSV tách riêng thông tin về màu sắc (Hue), độ bão hòa (Saturation) và độ sáng (Value), giúp việc xử lý màu sắc trở nên thuận tiện hơn. Để thực hiện chuyển ảnh từ không gian màu RGB sang HSV, đầu tiên, ảnh được chuẩn hóa về khoảng từ 0 đến 1 bằng câu lệnh: `img = img.astype(np.float32) / 255.0`. Việc chuẩn hóa giúp các phép tính đơn giản hơn vì công thức HSV được xây dựng trên các giá trị nằm trong khoảng này.

Sau đó chương trình xác định giá trị lớn nhất và nhỏ nhất trong ba kênh màu:

```
Cmax = np.maximum.reduce([R, G, B])
```

```
Cmin = np.minimum.reduce([R, G, B])
```

```
delta = Cmax - Cmin
```

Ý tưởng của bước này là Cmax biểu thị độ sáng của pixel nên được dùng để tính thành phần V. Hiệu số delta cho biết mức độ khác nhau giữa các màu, từ đó xác định độ bão hòa S. Kênh màu nào có giá trị lớn nhất sẽ quyết định sắc độ màu H.

Ví dụ, nếu kênh Red có giá trị lớn nhất thì màu sắc của pixel sẽ nằm gần vùng màu đỏ trên vòng tròn màu; tương tự đối với Green và Blue.

Sau khi tính được H, S và V, chương trình ghép ba thành phần lại để tạo ảnh HSV:
`hsv = np.stack([H, S, V], axis=-1).`

HSV → RGB

Để chuyển ảnh từ không gian màu HSV về RGB, đầu tiên chương trình tách ba thành phần:

`H = hsv[:, :, 0]`

`S = hsv[:, :, 1]`

`V = hsv[:, :, 2]`

Từ các giá trị này, chương trình tính các giá trị trung gian:

`C = V * S`

`X = C * (1 - np.abs((H / 60) % 2 - 1))`

`m = V - C`

Ý tưởng của bước này là xác định màu sắc thực tế dựa trên vị trí của Hue trên vòng tròn màu. Tùy theo khoảng giá trị của H, chương trình sẽ phân bố cường độ màu cho ba kênh R, G và B bằng các điều kiện tương ứng.

Sau khi có giá trị R, G và B, chương trình ghép chúng lại thành ảnh màu:

`rgb = np.stack( [(R + m), (G + m), (B + m)], axis=-1)`

Cuối cùng, giá trị màu được đưa trở lại khoảng từ 0 đến 255 để phù hợp với cách lưu trữ ảnh RGB thông thường. Kết quả thu được là ảnh màu gần giống với ảnh ban đầu trước khi chuyển sang HSV: `rgb = np.clip(rgb * 255, 0, 255).`

3.5.2. Phương pháp OpenCV



Hình 11. Kết quả chuyển đổi giữa các không gian màu RGB, Gray và HSV bằng OpenCV

RGB → Gray

OpenCV cung cấp sẵn hàm ***cv2.cvtColor()*** để chuyển đổi giữa các không gian màu. Việc chuyển ảnh RGB sang ảnh xám được thực hiện bằng câu lệnh:

```
gray_cv = cv2.cvtColor( img_np, cv2.COLOR_RGB2GRAY )
```

Hàm này tự động áp dụng công thức chuyển đổi từ RGB sang Gray mà không cần phải tách riêng các kênh màu như phương pháp thủ công. Kết quả thu được là ảnh chỉ chứa thông tin về độ sáng của từng pixel.

RGB → HSV

Việc chuyển đổi từ không gian màu RGB sang HSV được thực hiện bằng câu lệnh:

```
hsv_cv = cv2.cvtColor( img_np, cv2.COLOR_RGB2HSV )
```

OpenCV tự động tính toán ba thành phần Hue (H), Saturation (S) và Value (V) dựa trên giá trị của các kênh màu RGB. Nhờ đó, chương trình không cần tự tính các giá trị lớn nhất, nhỏ nhất hay độ chênh lệch giữa các kênh màu như ở phương pháp thủ công.

Kết quả thu được là ảnh trong không gian màu HSV, trong đó:

- H biểu diễn sắc độ màu.
- S biểu diễn độ bão hòa màu.
- V biểu diễn độ sáng.

HSV → RGB

Để chuyển ảnh từ không gian màu HSV trở lại RGB, chương trình sử dụng câu lệnh:

```
rgb_recovered_cv = cv2.cvtColor(hsv_cv, cv2.COLOR_HSV2RGB)
```

OpenCV sẽ tự động thực hiện toàn bộ quá trình chuyển đổi và khôi phục lại ba kênh màu Red, Green và Blue. Kết quả thu được là ảnh RGB gần giống với ảnh ban đầu trước khi chuyển sang HSV.

3.5.3. So sánh phương pháp thủ công và OpenCV

```

===== RGB -> GRAY =====
Manual : 0.000999 giây
OpenCV : 0.002002 giây

===== RGB -> HSV =====
Shape Manual : (150, 150, 3)
Shape OpenCV : (150, 150, 3)
Manual : 0.002003 giây
OpenCV : 0.000996 giây

===== HSV -> RGB =====
Manual : 0.003001 giây
OpenCV : 0.002002 giây

```

Hình 12. Kết quả so sánh thời gian thực thi và số pixel khác nhau giữa phương pháp thủ công và OpenCV

Kết quả thực nghiệm cho thấy cả phương pháp thủ công và OpenCV đều thực hiện thành công các phép chuyển đổi màu sắc RGB $\rightarrow$ Gray, RGB $\rightarrow$ HSV và HSV $\rightarrow$ RGB.

Đối với phép chuyển đổi RGB $\rightarrow$ Gray, hai ảnh kết quả gần như giống nhau khi quan sát bằng mắt thường. Điều này cho thấy công thức chuyển đổi ảnh xám được cài đặt thủ công đã hoạt động chính xác và cho kết quả tương đương với hàm `cv2.cvtColor()` của OpenCV.



Hình 13. Ảnh so sánh kết quả chuyển đổi RGB $\rightarrow$ Gray giữa phương pháp thủ công và OpenCV

Đối với phép chuyển đổi HSV $\rightarrow$ RGB, ảnh được khôi phục từ hai phương pháp cũng có chất lượng rất tương đồng. Màu sắc, độ sáng và nội dung ảnh được bảo toàn tốt sau

quá trình chuyển đổi. Điều này chứng tỏ quá trình chuyển đổi $RGB \rightarrow HSV$ và $HSV \rightarrow RGB$ trong phần cài đặt thủ công đã được thực hiện đúng nguyên lý.



Hình 14. Ảnh so sánh kết quả chuyển đổi $HSV \rightarrow RGB$ giữa phương pháp thủ công và OpenCV

Về thời gian xử lý, có thể thấy thời gian thực thi của hai phương pháp đều rất nhỏ do ảnh thử nghiệm có kích thước không lớn (150×150 pixel). Trong phép chuyển đổi $RGB \rightarrow Gray$, phương pháp thủ công thậm chí cho thời gian thấp hơn OpenCV. Tuy nhiên, sự chênh lệch này rất nhỏ và có thể bị ảnh hưởng bởi nhiều yếu tố như thời điểm đo, tài nguyên hệ thống hoặc chi phí gọi hàm của thư viện.

Đối với các phép chuyển đổi phức tạp hơn như $RGB \rightarrow HSV$ và $HSV \rightarrow RGB$, OpenCV cho tốc độ xử lý nhanh hơn. Điều này cho thấy lợi thế của thư viện OpenCV khi các thuật toán đã được tối ưu hóa ở mức thấp, giúp tăng hiệu năng xử lý đối với các bài toán xử lý ảnh.

Nhìn chung, phương pháp thủ công giúp hiểu rõ nguyên lý chuyển đổi giữa các không gian màu và cách biểu diễn màu sắc trong ảnh số. Trong khi đó, OpenCV giúp giảm số lượng mã lệnh, dễ sử dụng hơn và phù hợp với các ứng dụng thực tế yêu cầu hiệu năng cao.

3.6. Điều chỉnh độ sáng

3.6.1. Phương pháp thủ công

Phương pháp thủ công

Để điều chỉnh độ sáng của ảnh, chương trình xây dựng hàm `adjust_brightness_manual(img, beta)`. Ý tưởng của phương pháp này là thay đổi giá trị của tất cả các pixel trong ảnh bằng cách cộng hoặc trừ một hằng số β .

Trong hàm: `result = img.astype(np.int32) + beta`

Ảnh ban đầu được chuyển sang kiểu dữ liệu `int32` trước khi thực hiện phép cộng. Việc này giúp tránh hiện tượng tràn số khi giá trị pixel vượt ra ngoài phạm vi lưu trữ của kiểu `uint8` (0–255).

Tham số β quyết định mức thay đổi độ sáng:

- $\beta > 0$: tăng độ sáng của ảnh.
- $\beta < 0$: giảm độ sáng của ảnh.

Ví dụ:

- Nếu một pixel có giá trị 100 và $\beta = 50$ thì giá trị mới sẽ là 150.
- Nếu một pixel có giá trị 100 và $\beta = -50$ thì giá trị mới sẽ là 50.

Sau khi thay đổi độ sáng, chương trình sử dụng `result = np.clip(result, 0, 255)` để giới hạn giá trị pixel trong khoảng từ 0 đến 255. Nếu không giới hạn, các giá trị vượt quá phạm vi sẽ gây lỗi hoặc làm ảnh hiển thị không chính xác.

Cuối cùng, ảnh được chuyển trở lại kiểu dữ liệu `uint8` để phù hợp với định dạng lưu trữ ảnh thông thường.

Trong thí nghiệm, chương trình thực hiện ba mức điều chỉnh độ sáng: -50, 50 và 100. Kết quả cho thấy khi giá trị β tăng, toàn bộ ảnh trở nên sáng hơn do giá trị của các pixel được tăng lên. Ngược lại, khi β âm, ảnh trở nên tối hơn do giá trị pixel bị giảm xuống. Đây là phương pháp điều chỉnh độ sáng đơn giản và dễ hiểu nhất trong xử lý ảnh số.



Hình 15. Kết quả điều chỉnh độ sáng ảnh với các giá trị β khác nhau bằng phương pháp thủ công

3.6.2. Phương pháp OpenCV

Để điều chỉnh độ sáng ảnh bằng OpenCV, chương trình xây dựng hàm `adjust_brightness_cv(img, beta)` và sử dụng hàm có sẵn **`cv2.convertScaleAbs()`**: `return cv2.convertScaleAbs( img, alpha=1.0, beta=beta )`

Hàm `cv2.convertScaleAbs()` thực hiện phép biến đổi: $I' = \alpha I + \beta$

Trong đó:

- I là giá trị pixel ban đầu.
- I' là giá trị pixel sau biến đổi.
- α là hệ số tương phản.
- β là giá trị điều chỉnh độ sáng.

Trong bài thực hành, giá trị $\alpha = 1.0$ được giữ cố định nên độ tương phản của ảnh không thay đổi. Việc tăng hoặc giảm độ sáng được thực hiện thông qua tham số β .

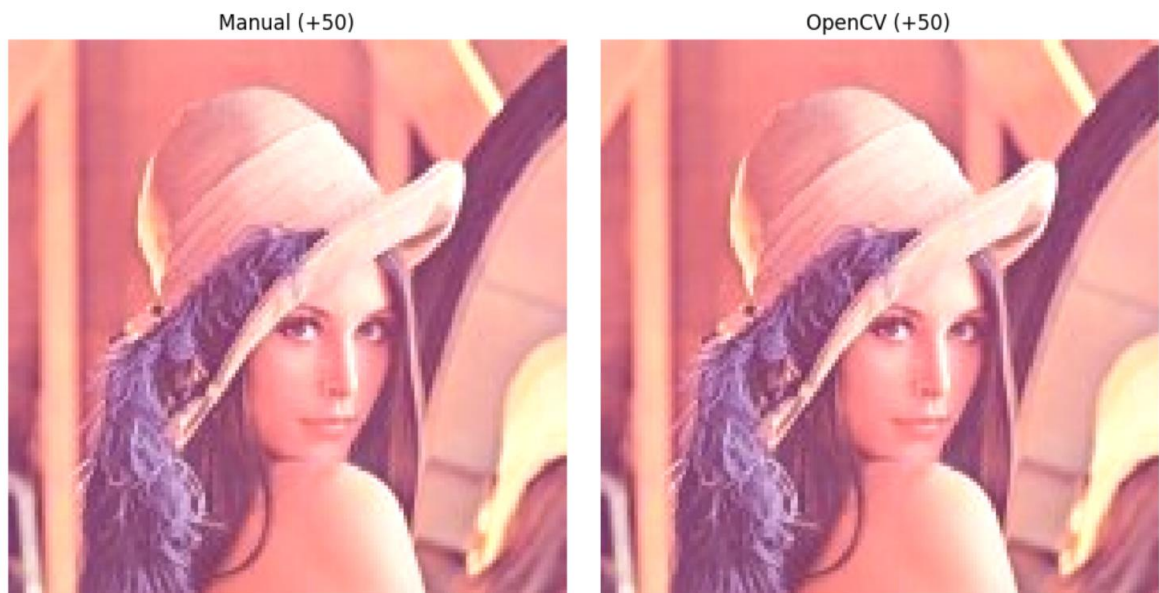
- Khi $\beta > 0$, giá trị pixel tăng lên làm ảnh sáng hơn.
- Khi $\beta < 0$, giá trị pixel giảm xuống làm ảnh tối hơn.

Sau khi xây dựng hàm, chương trình thực hiện ba mức điều chỉnh độ sáng: -50, 50 và 100. Khác với phương pháp thủ công, OpenCV tự động xử lý việc giới hạn giá trị pixel trong khoảng từ 0 đến 255. Nhờ đó chương trình không cần sử dụng thêm hàm `np.clip()` để kiểm tra giá trị vượt ngưỡng. Mã lệnh ngắn gọn hơn, dễ sử dụng hơn và tốc độ xử lý cũng được tối ưu bởi thư viện OpenCV.



Hình 16. Kết quả điều chỉnh độ sáng ảnh với các giá trị β khác nhau bằng phương pháp OpenCV

3.6.3. So sánh hai phương pháp



Hình 17. So sánh kết quả điều chỉnh độ sáng ($\beta = +50$) giữa phương pháp thủ công và OpenCV

Kết quả thực nghiệm cho thấy phương pháp thủ công và OpenCV đều thực hiện thành công phép điều chỉnh độ sáng của ảnh với giá trị $\beta = +50$. Khi quan sát hai ảnh kết quả, có thể thấy hai ảnh thu được từ phương pháp thủ công và OpenCV gần như giống hệt nhau về màu sắc và chất lượng hiển thị.

```
===== SO SÁNH KÍCH THƯỚC =====
Manual : (150, 150, 3)
OpenCV : (150, 150, 3)

===== SO SÁNH TỐC ĐỘ =====
Manual : 0.000000 giây
OpenCV : 0.000000 giây
```

Hình 18. Kết quả so sánh kích thước và thời gian thực thi giữa phương pháp thủ công và OpenCV

Kết quả so sánh kích thước cho thấy cả hai phương pháp đều giữ nguyên kích thước ảnh. Điều này cho thấy phép điều chỉnh độ sáng chỉ tác động đến giá trị màu của pixel mà không làm thay đổi số hàng, số cột hay số kênh màu của ảnh.

Về thời gian thực thi, kết quả hiển thị là 0.000000 giây đối với cả hai phương pháp. Nguyên nhân là do ảnh thử nghiệm có kích thước nhỏ (150×150 pixel) nên thời gian xử lý thực tế rất ngắn và nhỏ hơn độ chính xác hiển thị của hàm đo thời gian. Vì vậy chưa thể đánh giá chính xác sự chênh lệch tốc độ giữa hai phương pháp từ thí nghiệm này.

Nhìn chung, phương pháp thủ công và OpenCV đều cho kết quả tương đương về mặt trực quan. Tuy nhiên, OpenCV có ưu điểm là mã nguồn ngắn gọn hơn, dễ sử dụng hơn và

được tối ưu sẵn cho các bài toán xử lý ảnh thực tế. Trong khi đó, phương pháp thủ công giúp hiểu rõ nguyên lý điều chỉnh độ sáng bằng cách cộng hoặc trừ một giá trị cố định vào các pixel của ảnh.

3.7. Điều chỉnh độ tương phản

3.7.1. Phương pháp thủ công

Phương pháp thủ công

Để điều chỉnh độ tương phản của ảnh, chương trình xây dựng hàm `adjust_contrast_manual(img, alpha)`. Ý tưởng của phương pháp này là nhân giá trị của tất cả các pixel với một hệ số alpha.

Trong hàm: `result = img.astype(np.float32) * alpha`, ảnh được chuyển sang kiểu dữ liệu float32 trước khi thực hiện phép nhân để đảm bảo độ chính xác của phép tính. Sau đó, mỗi giá trị pixel trong ảnh sẽ được nhân với hệ số alpha.

Ý nghĩa của tham số alpha như sau:

- $\alpha > 1$: tăng độ tương phản.
- $\alpha = 1$: giữ nguyên ảnh.
- $\alpha < 1$: giảm độ tương phản.

Ví dụ:

- Một pixel có giá trị 100:
 - Khi $\alpha = 0.5$, giá trị mới là 50.
 - Khi $\alpha = 1.5$, giá trị mới là 150.
 - Khi $\alpha = 2.0$, giá trị mới là 200.

Sau khi thực hiện phép nhân, một số pixel có thể vượt quá phạm vi cho phép của ảnh số (0–255). Vì vậy chương trình sử dụng: `result = np.clip(result, 0, 255)` để giới hạn các giá trị pixel trong khoảng từ 0 đến 255. Cuối cùng, ảnh được chuyển trở lại kiểu dữ liệu uint8 để phù hợp với định dạng lưu trữ ảnh thông thường: `return result.astype(np.uint8)`.

Trong thí nghiệm, chương trình thực hiện bốn mức điều chỉnh độ tương phản: 0.5, 1.0, 1.5, 2.0. Kết quả cho thấy khi giá trị alpha giảm xuống dưới 1, các vùng sáng và vùng tối trở nên gần giống nhau hơn nên ảnh trông nhạt và ít nổi bật. Ngược lại, khi alpha lớn hơn 1, sự khác biệt giữa vùng sáng và vùng tối được tăng lên làm ảnh rõ nét hơn. Tuy nhiên, nếu

alpha quá lớn, một số vùng sáng có thể bị bão hòa về giá trị 255 hoặc một số vùng tối bị mất chi tiết, làm giảm chất lượng hình ảnh.



Hình 19. Kết quả điều chỉnh độ tương phản ảnh với các hệ số α khác nhau bằng phương pháp thủ công

3.7.2. Phương pháp OpenCV

Để điều chỉnh độ tương phản của ảnh bằng OpenCV, chương trình xây dựng hàm `adjust_contrast_cv(img, alpha)` và sử dụng hàm có sẵn `cv2.convertScaleAbs()`: `return cv2.convertScaleAbs( img, alpha=alpha, beta=0)`

Hàm `cv2.convertScaleAbs()` thực hiện phép biến đổi: $I' = \alpha I + \beta$

Trong đó:

- I là giá trị pixel ban đầu.
- I' là giá trị pixel sau khi biến đổi.
- α (alpha) là hệ số điều chỉnh độ tương phản.
- β (beta) là giá trị điều chỉnh độ sáng.

Trong bài thực hành này, giá trị $\beta = 0$ được giữ cố định nên độ sáng của ảnh không thay đổi. Việc tăng hoặc giảm độ tương phản được thực hiện thông qua tham số alpha.

Khác với phương pháp thủ công, OpenCV tự động thực hiện phép nhân hệ số tương phản, chuyển đổi kiểu dữ liệu và giới hạn giá trị pixel trong khoảng từ 0 đến 255. Nhờ đó chương trình không cần sử dụng thêm các hàm như `astype()` hoặc `np.clip()` để xử lý dữ liệu.



Hình 20. Kết quả điều chỉnh độ tương phản ảnh với các hệ số α khác nhau bằng phương pháp OpenCV

3.7.3. So sánh hai phương pháp

Kết quả thực nghiệm cho thấy cả phương pháp thủ công và OpenCV đều thực hiện thành công phép điều chỉnh độ tương phản với hệ số $\alpha = 1.5$.

Như vậy, cả hai phương pháp đều giữ nguyên kích thước và số kênh màu của ảnh. Phép biến đổi chỉ tác động đến giá trị pixel mà không làm thay đổi cấu trúc dữ liệu của ảnh.



Hình 21. So sánh kết quả tăng độ tương phản với hệ số $\alpha = 1.5$ giữa phương pháp thủ công và OpenCV

Về mặt trực quan, ảnh thu được từ hai phương pháp gần như giống hệt nhau. Mức độ sáng, độ tương phản và các chi tiết trên ảnh không có sự khác biệt đáng kể khi quan sát bằng mắt thường. Điều này cho thấy hàm `cv2.convertScaleAbs()` của OpenCV đang thực hiện nguyên lý tương tự như phương pháp thủ công đã xây dựng.

```
===== SO SÁNH KÍCH THƯỚC =====
Manual : (150, 150, 3)
OpenCV : (150, 150, 3)

===== SO SÁNH TỐC ĐỘ =====
Manual : 0.003007 giây
OpenCV : 0.000000 giây
```

Hình 22. Kết quả so sánh kích thước và thời gian thực thi giữa phương pháp thủ công và OpenCV trong điều chỉnh độ tương phản

Kết quả đo thời gian thực thi cho thấy thời gian xử lý của OpenCV rất nhỏ nên được làm tròn về 0 khi hiển thị với độ chính xác hiện tại. Điều này cho thấy OpenCV được tối ưu tốt và có khả năng xử lý nhanh hơn so với phương pháp tự cài đặt bằng các phép toán trên mảng NumPy.

Từ kết quả thực nghiệm có thể kết luận rằng cả hai phương pháp đều cho kết quả tương đương về chất lượng ảnh. Tuy nhiên, OpenCV có ưu điểm về tốc độ và tính tiện lợi trong quá trình triển khai thực tế, trong khi phương pháp thủ công giúp hiểu rõ nguyên lý điều chỉnh độ tương phản thông qua hệ số alpha.

3.8. Nhận xét

Việc tự cài đặt các phép xử lý ảnh giúp hiểu rõ nguyên lý hoạt động của từng thuật toán và cách thao tác trực tiếp trên dữ liệu pixel. Tuy nhiên, phương pháp này yêu cầu nhiều bước xử lý và thời gian thực thi lớn hơn.

Trong khi đó, OpenCV cung cấp các hàm được tối ưu hóa sẵn, giúp giảm số lượng lệnh cần viết, tăng tốc độ xử lý và thuận tiện cho việc phát triển các ứng dụng thực tế.

TÀI LIỆU THAM KHẢO

- [1] OpenCV Documentation, *Geometric Transformations of Images*. Truy cập tại: <https://docs.opencv.org>
- [2] OpenCV Documentation, *Changing the Contrast and Brightness of an Image*. Truy cập tại: <https://docs.opencv.org>
- [3] OpenCV Documentation, *OpenCV-Python Tutorials*. Truy cập tại: <https://docs.opencv.org>
- [4] NumPy Documentation, *Indexing on ndarrays*. Truy cập tại: <https://numpy.org>
- [5] Wikipedia, *Grayscale*. Truy cập tại: <https://en.wikipedia.org/wiki/Grayscale>
- [6] Wikipedia, *HSL and HSV*. Truy cập tại: https://en.wikipedia.org/wiki/HSL_and_HSV
- [7] LearnOpenCV, *Image Resizing with OpenCV*. Truy cập tại: <https://learnopencv.com>
- [8] NumPy Developers, "NumPy User Guide - Indexing on ndarrays", <https://numpy.org/doc/stable/user/basics.indexing.html>